

Training Gradient-Boosted-Neuron Networks Without Backpropagation:

Research Goal and Solution Pathway

1 Research Goal

Build a neural network architecture in which each neuron is itself a gradient-boosted tree ensemble (XGBoost) rather than a linear-then-activation unit, and train the full network end-to-end **without backpropagation**, since trees are non-differentiable and the chain rule cannot pass through them. The objective is a boosting-native replacement for gradient-based credit assignment: a mechanism that determines how much each node contributed to the network’s output error, and updates each node accordingly, using only operations native to tree ensembles (fitting a tree to a target).

2 Solution Pathway

The following fixes assemble, in order, into the proposed training procedure.

Fix 1: A nonlinear meta-model in place of linear regression

After a forward pass produces intermediate node outputs $h_1, \dots, h_L$ and prediction $\hat{y}$, form the residual $r = y - \hat{y}$ and fit a **gradient-boosted** meta-model, not an OLS regression, to the meta-dataset $\{(h_{i1}, \dots, h_{iL}), r_i\}$:

$$\hat{r} = M(h_1, \dots, h_L), \quad M \text{ a boosted ensemble}$$

A boosted M captures the nonlinear structure of $E[r \mid h]$ that a linear fit cannot, since $h_1, \dots, h_L$ are themselves outputs of piecewise-constant, interaction-heavy trees.

Fix 2: Per-node, per-observation credit via SHAP decomposition

Since M is a tree ensemble, TreeSHAP gives an exact, efficiently computable additive decomposition of its output:

$$M(h_i) = \phi_0 + \sum_{j=1}^L \phi_{ij}$$

ϕ_{ij} is used as node j ’s assigned share of observation i ’s residual, a genuine per-observation credit-assignment signal rather than an aggregated importance score.

Fix 3: A boosting-native node update rule

Each node j is updated by extending its own boosting sequence with a new tree fit directly to its assigned credit:

$$h_j \longleftarrow h_j + \nu \cdot T_j(\{x_i, \phi_{ij}\})$$

where T_j denotes a newly fit tree and ν a small boost rate. This is a backprop-free update composed entirely of the native operation of a tree ensemble: fit a tree to a target.

Fix 4: Cross-fitting to keep the attribution valid

M and its SHAP decomposition are fit on out-of-fold node outputs h_i , rather than the in-sample outputs the nodes just produced on the same training pass, so that the credit signal ϕ_{ij} is not optimistically inflated by the ensembles having already been optimized against it.

Fix 5: Causal / asymmetric Shapley values, respecting the known layer order

Because the architecture has no skip connections, the layer sequence is a known causal (Markov) ordering: h_{j+1} depends only on h_j . Rather than symmetric Shapley values, which split credit across all orderings and smear it across correlated adjacent nodes, asymmetric Shapley values restrict credit-splitting to permutations consistent with this known order, so an earlier node is not credited for effects that flow only through a later one.

Fix 6: Local, chained attribution instead of one global regression

Rather than fitting a single global $M(h_1, \dots, h_L)$ across all layers jointly, propagate credit backward one adjacent pair at a time:

$$M_L : r \sim h_L \longrightarrow \text{credit}_{L-1} \sim h_{L-1} \longrightarrow \dots \longrightarrow \text{credit}_1 \sim h_1$$

Each step involves only two adjacent, causally-ordered variables, mirroring how the chain rule computes $\partial \mathcal{L} / \partial h_j$ from only the immediate neighbor $\partial \mathcal{L} / \partial h_{j+1}$, and avoiding the multicollinearity that a single joint fit across all L layers would introduce.

Fix 7: An outer corrective loop

After updating node j , every downstream h changes, since it was computed from node j 's prior output. The full procedure therefore loops: update a node, recompute the forward pass, refit the meta-model(s), recompute the credit signal, and repeat, in the same spirit as GrowNet's corrective step, which jointly re-fits all prior weak learners after each new one is added rather than freezing them permanently.

Fix 8: Grounding the update target in a loss-descent guarantee

TreeSHAP's discrete combinatorial value guarantees only that ϕ_{ij} is a *fair* allocation of credit (efficiency and permutation symmetry, per Shapley's axiomatic characterization). It does not by itself guarantee that updating h_j in the direction of ϕ_{ij} decreases the network's loss, the property ordinary functional gradient descent relies on. The principle that supplies that stronger guarantee is the **Aumann–Shapley value**, the path-integral generalization of the Shapley value to differentiable settings,

$$\phi_j = \int_0^1 \frac{\partial M}{\partial h_j} (h' + \alpha(h - h')) d\alpha,$$

which collapses exactly to the true gradient $\partial M/\partial h_j$ in the differentiable limit, and underlies Integrated Gradients. The fix: treat plain TreeSHAP credit as a fairness heuristic only, and validate the update rule (Fix 3) empirically, checking on held-out data whether ϕ_{ij} -directed node updates actually decrease loss, or move toward a path-integrated attribution consistent with Aumann–Shapley, before trusting it as a true substitute for a gradient.

3 Assembled Procedure

```

repeat until convergence:
  forward pass  $\rightarrow h_1, \dots, h_L, \hat{y}$ 
   $r_i = y_i - \hat{y}_i$ 
  for  $j = L$  down to 1:      # Fix 6: local, chained, not global
    fit boosted  $M_j$ : current credit target  $\sim h_j$     # Fixes 1, 5: nonlinear, order-respecting
     $\phi_{\cdot j} \leftarrow \text{TreeSHAP}(M_j)$     # Fix 2: per-observation credit
    validate  $\phi_{\cdot j}$  against held-out loss change    # Fix 8: loss-descent check
     $h_j \leftarrow h_j + \nu T_j(\{x_i, \phi_{ij}\})$     # Fix 3: boosting-native update
    (fit  $M_j$  on out-of-fold  $h_j$ )    # Fix 4: cross-fitting
  recompute forward pass with updated nodes    # Fix 7: corrective loop

```